

LESSON GUIDE

Technology Plus

Unit 2: Programming

4.2.1 – Fundamental Data Types

Lesson Overview:

In this lesson, students will investigate fundamental data types through simple code examples.

Students will:

- Identify fundamental data types.
- Predict outputs based on common operations and data types.

Guiding Question: What types of data are built-in to our programming languages, and what can we do with them?

Suggested Grade Levels: 8-12

Technology Needed: No

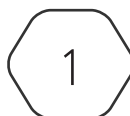
Approximate Time Required: 45-60 minutes

Standards:

CompTIA Tech+ FC0-U71 Objective

- 4.1 – Compare and contrast programming language categories.
 - Char
 - Strings
 - Numbers
 - Integers
 - Floats
 - Boolean

This content is based upon work supported by the US Department of Homeland Security's Cybersecurity & Infrastructure Security Agency under the Cybersecurity Education Training and Assistance Program (CETAP).



CYBER.ORG
THE ACADEMIC INITIATIVE OF THE CYBER INNOVATION CENTER

Copyright © 2025 Cyber Innovation Center
All Rights Reserved. Not for Distribution.

Lesson Background

Background Information:

In programming, different data types are used to represent different kinds of information. While the exact set of data types can vary depending on the language, most languages share some common categories:

- **char** represents a single character.
- **string** represents a sequence of characters.
- **integers** represent whole numbers.
- **floats** and **doubles** represent numbers with decimal values.
- **fixed-point** numbers represent decimals with a set precision.
- **Boolean** values are either *true* or *false*, often stored internally as 1 or 0.

Programmers may also use casting to convert data from one type to another, such as turning a string into an integer or a float into a string. Understanding how data types work is essential, since computers process different kinds of information in different ways.

Materials Needed:

Materials per Class	• Lesson Guide • Lesson Slides 4.2.1 – Fundamental Data Types • Activity Slides 4.2.1 – I Have/Who Has... Fun with Data • Activity 4.2.1 – I Have/Who Has... Fun with Data cards
Materials per Student	• Student Notes 4.2.1 – Fundamental Data Types • Student Handout 4.2.1 – Fundamental Data Types • Assessment 4.2.1 – Fundamental Data Types (optional)

Lesson Background

Cyber Terms:

Term	Definition
fundamental	primitive or built-in
char	a single character, such as a letter, digit, or symbol
string	a sequence of characters, including letters, numbers, or symbols
index	a number representing a position within a sequence
integer	a whole number without a decimal point (e.g., -3, 0, 42); commonly written as int
float	a number that includes a decimal point (e.g., 3.141592, -0.01)
double	a more precise float (e.g., 3.141592653589)
fixed-point	a decimal number with a fixed number of digits after the decimal point (e.g., \$12.34 always has two decimal places)
Boolean	a value that is either true or false, often represented as 1 (true) or 0 (false)
casting	the process of converting one data type into another, such as turning a string into an integer or a float into a string

4.2.1 – Fundamental Data Types

Guiding Question: What types of data are built-in to our programming languages, and what can we do with them?

Lesson Launch (5-10 minutes)

- Distribute the Student Handout 4.2.1 – Fundamental Data Types and have students fill in the answers during the lesson.
- **Ask** students to analyze the picture on slide 3 and share observations. As they share and record their observations, remind them to be as specific as possible about what they see. Instead of saying ‘eggs’, students should say ‘6 eggs’ or ‘1 carton’.
- After students have had a minute or two to record their answers, **click** to switch images. Emphasize that the data we work with every day comes in different forms and can be represented in different ways.
- We will reference the grocery store image as we work today to explore how we store, interpret, and use data in a program.

Data: Putting the Fun in Fundamental (15-20 minutes)

- **Ask** students what they think of when they hear the word fundamental?
- Explain that **fundamental** in this sense means built-in or standard.
- Fundamental data types are those that pre-exist in a programming language, nothing needs to be imported or written by the user. Sometimes they are referred to as primitive data types.
- What a Character
 - **Char**: a single character
 - Can be a letter, digit, symbol (ASCII and Unicode represent characters)
 - Grocery Store example: aisle (for small store), symbol for price (\$)
- **String** it Together
 - A sequence of characters
 - Grocery Store example: names of products, names of employees
- An **index** is a number representing a position within the string
- String manipulation with built-in functions and expressions:
 - “Bread”.lower() → bread, changes all characters in a string to lowercase
 - “Bread”.upper() → BREAD, changes all characters in a string to uppercase
 - “Bread”.swapcase() → bREAD, switches the case of each string to its opposite (lowercase becomes uppercase; no changes to symbols or numbers)
 - “Bread”[0] → B

- "Bread"[2:] → ead
- "Bread"[:2] → Br
- Numbers
 - Integers
 - **int**: a whole number with no decimal point
 - Grocery Store example: quantity of an item, number of total items sold in store, number of staff
 - Floating Points
 - **float**: a number that can include decimal points
 - Grocery Store example: price, discount
 - Used for quick calculations or when speed matters (graphics)
 - Doubles
 - **double**: a floating-point number that can store larger and more accurate numbers
 - Used when precision matters in calculations
 - Fixed-point
 - **fixed-point**: a decimal number with a fixed number of digits after the decimal point
 - Often used for financial data where exact precision is important (e.g., \$12.34 always has two decimal places)
 - The dollar sign is included here just to illustrate that fixed-point means you can lock in how many decimal places are allowed. With money, that's two digits after the decimal (e.g., \$3.14). Students should understand that the fixed-point type itself isn't tied to currency, but that finance is a common real-world example where accuracy matters.
- **Boolean**
 - A value that is either True or False
 - Grocery Store example: does the store have bread in stock?
 - Booleans help create logic in programs.
 - Logical Operators: the output of a logical operator is a Boolean.
 - **4 == (7 - 3)** returns True
 - **True and False** returns False
 - Conditionals (branching): introduces a decision point in code based on a condition
 - **if 4 == (7 - 3):**
 print("The math works!")
 - Loops: repeat until a condition is met
 - **while True:**

```
print("Hello!")
```

- More on Branching and Loops in 4.4.1 – Programming Organization
- **Casting** Call
 - **cast**: the process of converting one data type into another
 - **int**("45") → 45
 - **str**(45) → "45"
 - **int**(4.4) → 4
 - Why do we need this?
 - Sometimes a number is treated as a number (age), but sometimes a number acts as a String (SSN).
 - Built-in functions, like input() which gathers input from a user, defaults as a string.

Activity: Fun Data (20-30 minutes)

- Before class, print and cut out the I Have/Who Has... Fun with Data activity cards and shuffle them well.
- Display Activity Slides 4.2.1 – I Have/Who Has... Fun with Data to guide students through the rules of the game.
- The "I have..." and "Who has..." prompts have been listed on the Student Handout so students can follow along, check their answers, or review the sequence before/after play.
- Decide whether students should complete the handout in advance or fill in answers as the game is played.
- Distribute all cards evenly among students. Because the game forms a closed loop, leaving out any card will break the sequence.
- Select one student to begin. That student reads their card aloud as in the example round on the slides.
- The student with the matching answer continues play by reading their card. Game play proceeds until the loop is completed, with the starting student's "I have..." closing the sequence.

Putting It All Together (5 minutes)

- What is the difference between a character and a string?

A character is a single letter, digit, or symbol while a string is a sequence of characters.
- When would you want to use a float instead of an integer?

A float is useful when you need to represent decimal places, such as in a price.
- What is a Boolean? How does it support control within a program?

A Boolean is a value that can be either True or False. It supports program control because it is the return value of logic statements and directs program flow based on its truth value.
- What is the expected output of print(4/int(2.3))? Why?

The expected output is 2.0 because 2.3 is being cast into an integer prior to division happening, making the actual division problem 4/2.